

Verteilte Systeme

Vorlesung im 3. Studienjahr (6. Semester)
an der DHBW Stuttgart

STG-TINF23BD

Dr. Christian Wiedmann

SS 2026

Zeitplan - 2026

Kurs	Datum	Uhrzeit	Ort	Themen
1	17.02.	08:45 – 12:00	A4.09	Kennenlernen, Vorlesungsüberblick, Einführung in verteilte Systeme
2	24.02.	08:45 – 12:00	A4.09	
3	03.03.	08:45 – 12:00	A4.09	
4	10.03.	08:45 – 12:00	A4.09	
5	17.03.	08:45 – 12:00	A4.09	Ankündigung Programmentwurf
6	24.03.	08:45 – 12:00	A4.09	Entfällt
7	31.03.	08:45 – 12:00	A4.09	Online
8	07.04.	08:45 – 12:00	A4.09	Online
9	14.04.	08:45 – 12:00	A4.09	Abgabe Programmentwurf
10	21.04.	08:45 – 12:00	A4.09	

Themenüberblick (dynamisch!)

Grundlagen

- Vorlesungsüberblick
- Einführung in verteilte Systeme

Architekturprinzipien

- Client-Server
- Peer-to-Peer
- Blockchain
- Microservices
- REST
- Event-driven

Kommunikation

- Synchron (Request/Response)
- Asynchron (Messaging, PubSub) ←

Parallelität, Zustand & Ausfälle

- Nebenläufigkeit
- Konsistenz (CAP)
- Fehlertoleranz
- Konsensus ←

Praxisanwendungen, Plattformen & Paradigmen

- Cloud-native Entwicklung, Orchestrierung, Betrieb (CI/CD, Kubernetes, Observability)
- Spark (Data Processing)
- Kafka (Event Streaming)
- BitTorrent (Peer-to-Peer)
- Bitcoin/Ethereum (Blockchain)

Sonstige Themen

- API-Gateways
- Edge Computing
- KI als Anwendungsfall

Kommunikation in verteilten Systemen: Asynchrone Kommunikation

Kommunikation in verteilten Systemen

Synchron (Request-Response)	Asynchron (Messaging)
Client wartet blockierend auf eine Antwort	Entkoppelt, zeitlich versetzt, nicht blockierend
• REST (HTTP-basiert) • RPC - Remote Procedure Call (gRPC) Früher: Java RMI, Apache Thrift, Corba • SOAP • Direkte Socket-Kommunikation	• Message Broker (z.B. RabbitMQ) • Message Bus / Event Streaming (z.B. Kafka)

Heute

- 1) Grundlagen „Message Queuing“
- 2) Message Broker (RabbitMQ)
- 3) Konsensus (Raft)
- 4) Event streaming (Kafka)

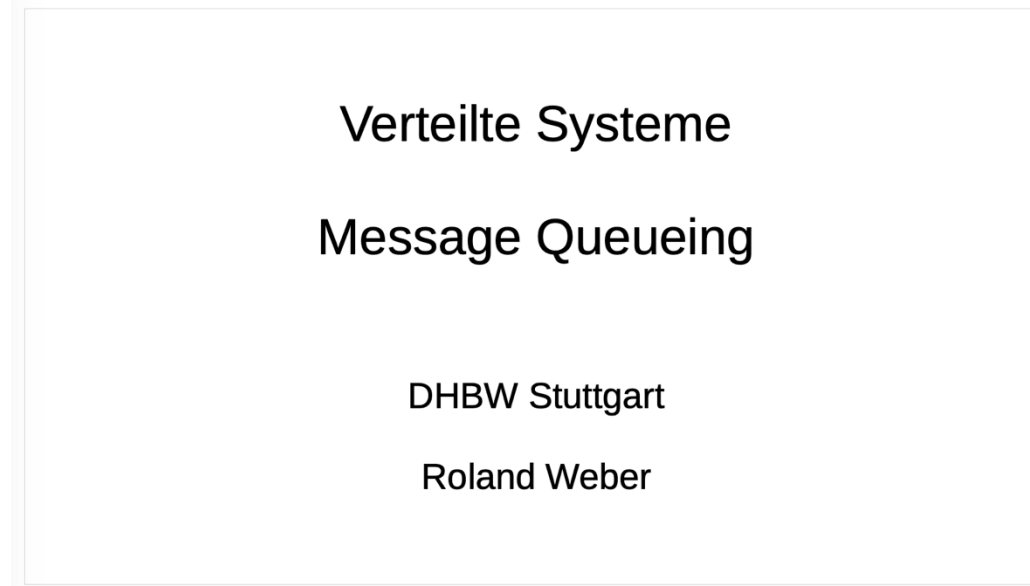
Grundlagen: Message Queues

Grundbegriffe

- **Message**
 - Payload (Bytestream, z.B. JSON)
 - Metadaten (z.B. Routing-Informationen)
- **Queue/Topic**
 - Puffer für Nachrichten
- **Producer/Consumer**
 - Sendet/empfängt Nachrichten
- **Source/Sink (Quelle/Senke)**
 - Sendet/empfängt Daten

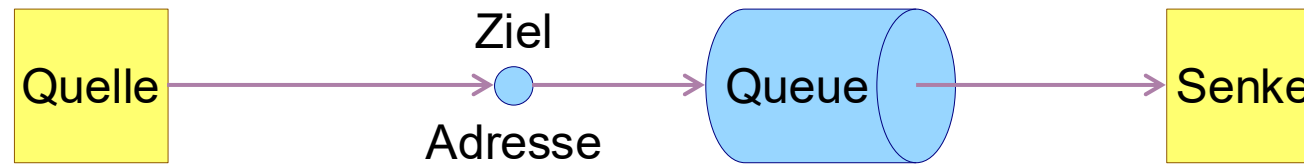
Message Queuing

- Angelehnt an Präsentation von Rolf Weber (DHBW Stuttgart, Verteilte Systeme)



Es gibt viele Message Queueing Systeme mit unterschiedlichen Protokollen. Die im Folgenden beschriebenen Interaktionen dürften sich mit fast allen davon implementieren lassen.

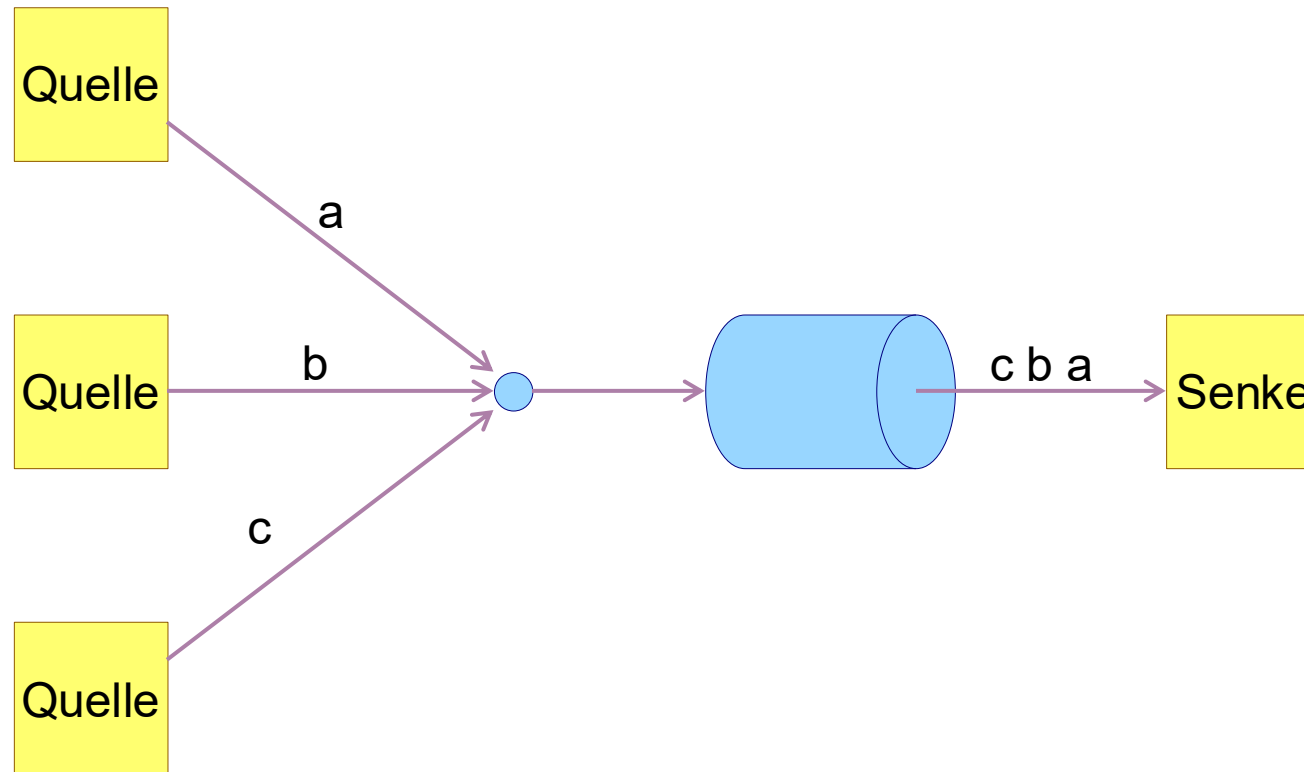
Point to point



Message Queue ist immer FIFO. Eine Quelle (Sender) schickt Nachrichten an ein Ziel, d.h. eine Adresse. Das Message Queueing System (MQS) bestimmt anhand des Ziels, in welche Queue die Nachricht geht. Eine Senke (Empfänger) erhält Nachrichten aus einer Queue. Jede Nachricht geht von einem Punkt (Sender) zu einem anderen Punkt (Empfänger).

Beispiele: Auftragsabwicklung (Order Processing), E-Mail

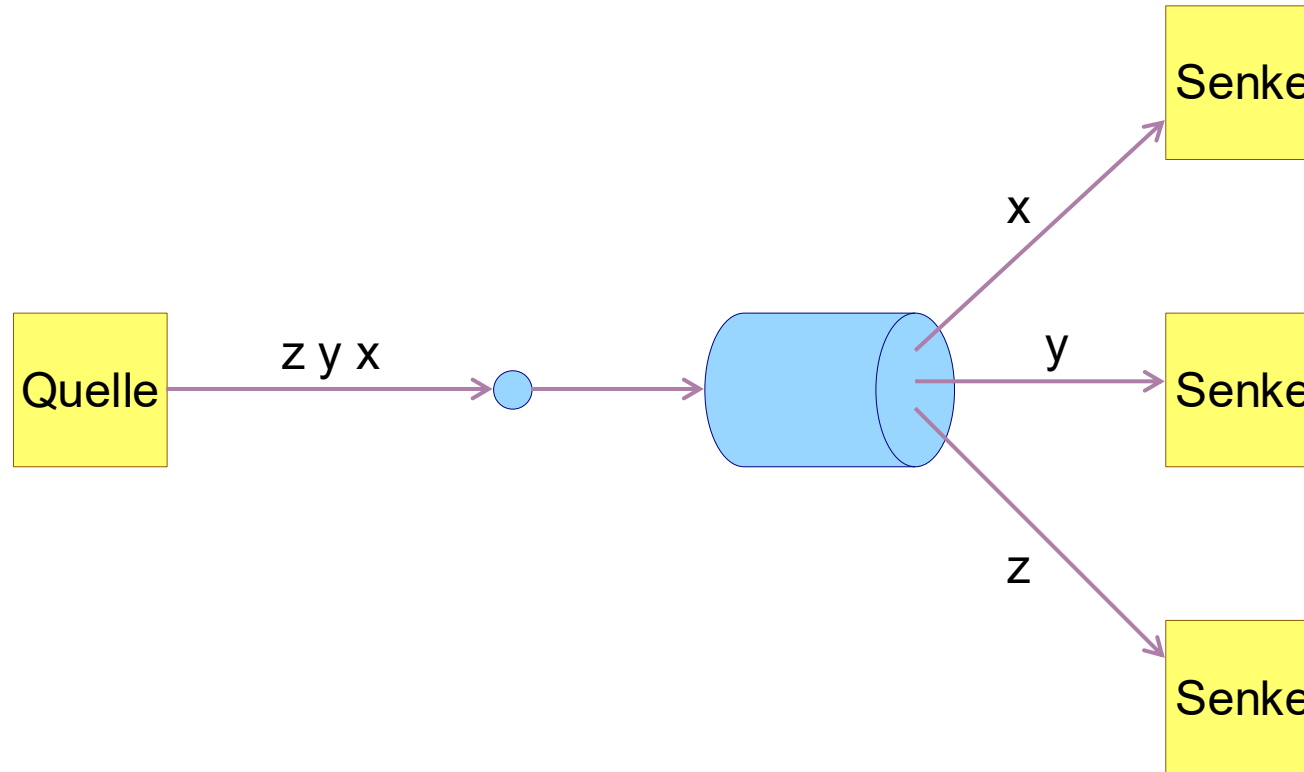
Point to point (multiple producers)



Alle Nachrichten an das gleiche Ziel landen in der gleichen Queue.

Beispiele: Web Shop, Mobile App, Telefon-Hotline

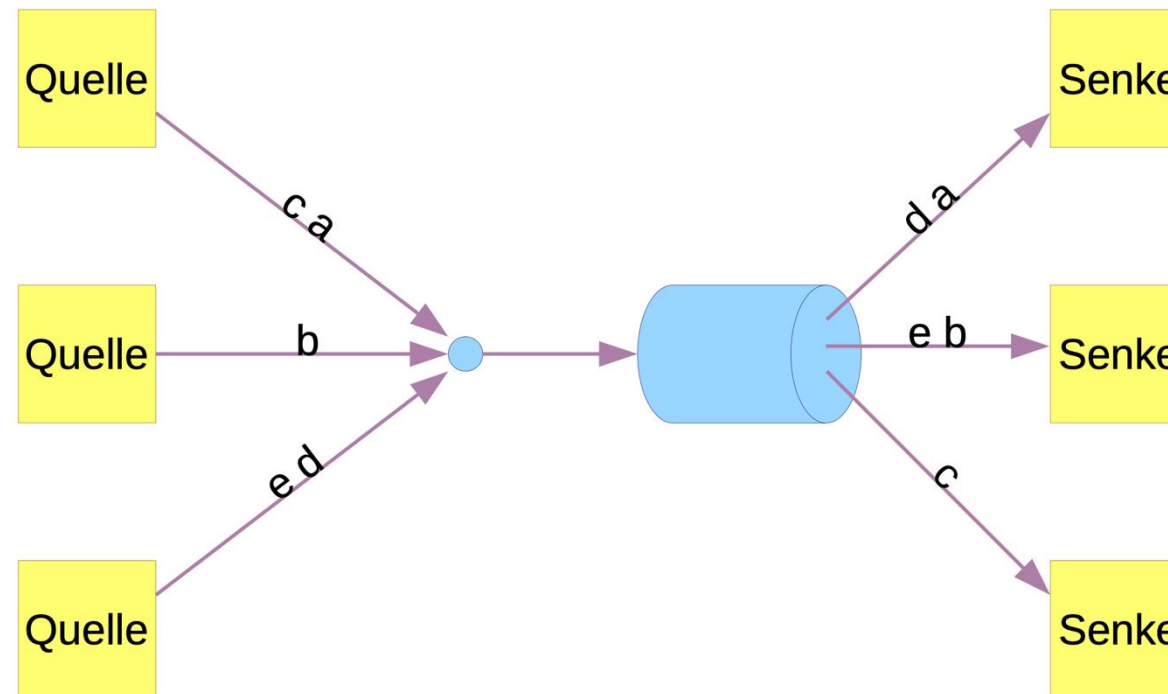
Point to point (multiple consumers)



Bei mehreren Empfängern an einer Queue geht jede Nachricht an einen davon.

Beispiel: Lastverteilung, Web-Shop: Versandabteilung

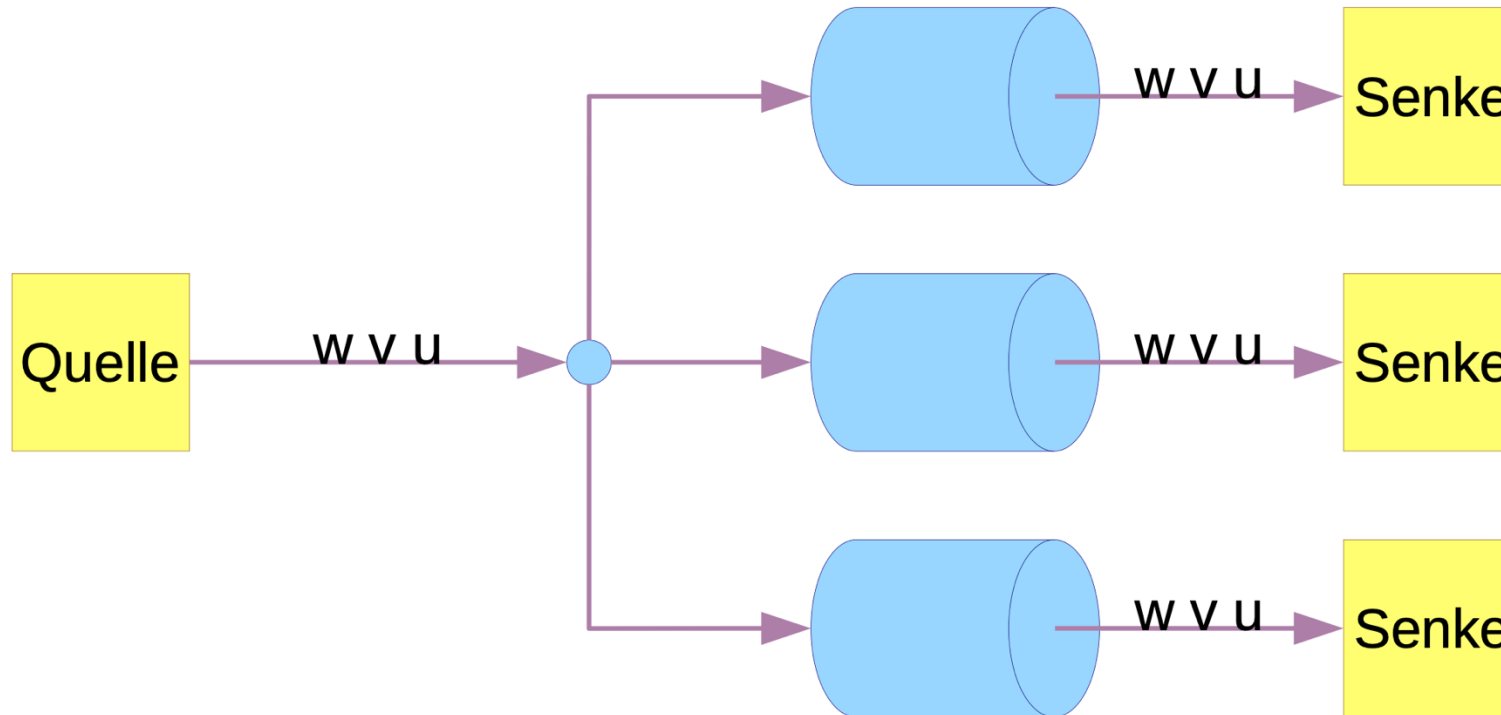
Point to point (multiple producers & consumers)



Jede Nachricht an das Ziel geht an genau einen der Empfänger an der zugehörigen Queue. Das MQS kann Load Balancing betreiben, indem es die Nachrichten gleichmäßig unter den Empfängern verteilt.

Beispiel: **Web-Shops, Order Queue, Versandabteilung**

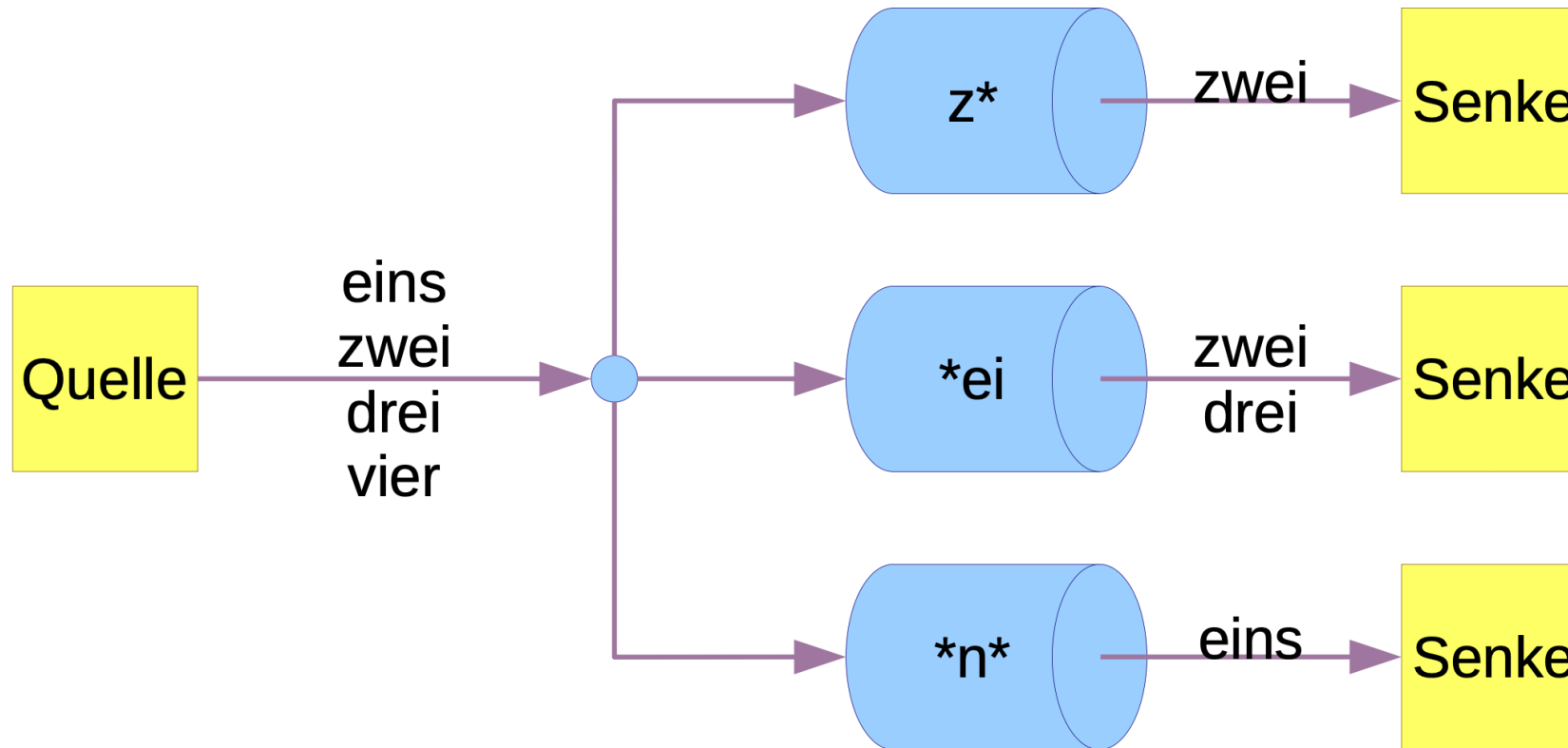
Publish / Subscribe (Fan-Out)



Jede Nachricht an das Ziel geht an jeden Empfänger, der seine Queue an das Ziel geknüpft hat. Der Sender einer Nachricht braucht nicht zu wissen, wieviele Empfänger sie erhalten. Bzw. wieviele Queues gerade mit dem Ziel verknüpft sind.

Beispiel: Mehrere Services sind an Bestellung interessiert (Versand-Service, Rechnungs-Service, E-Mail-Service)

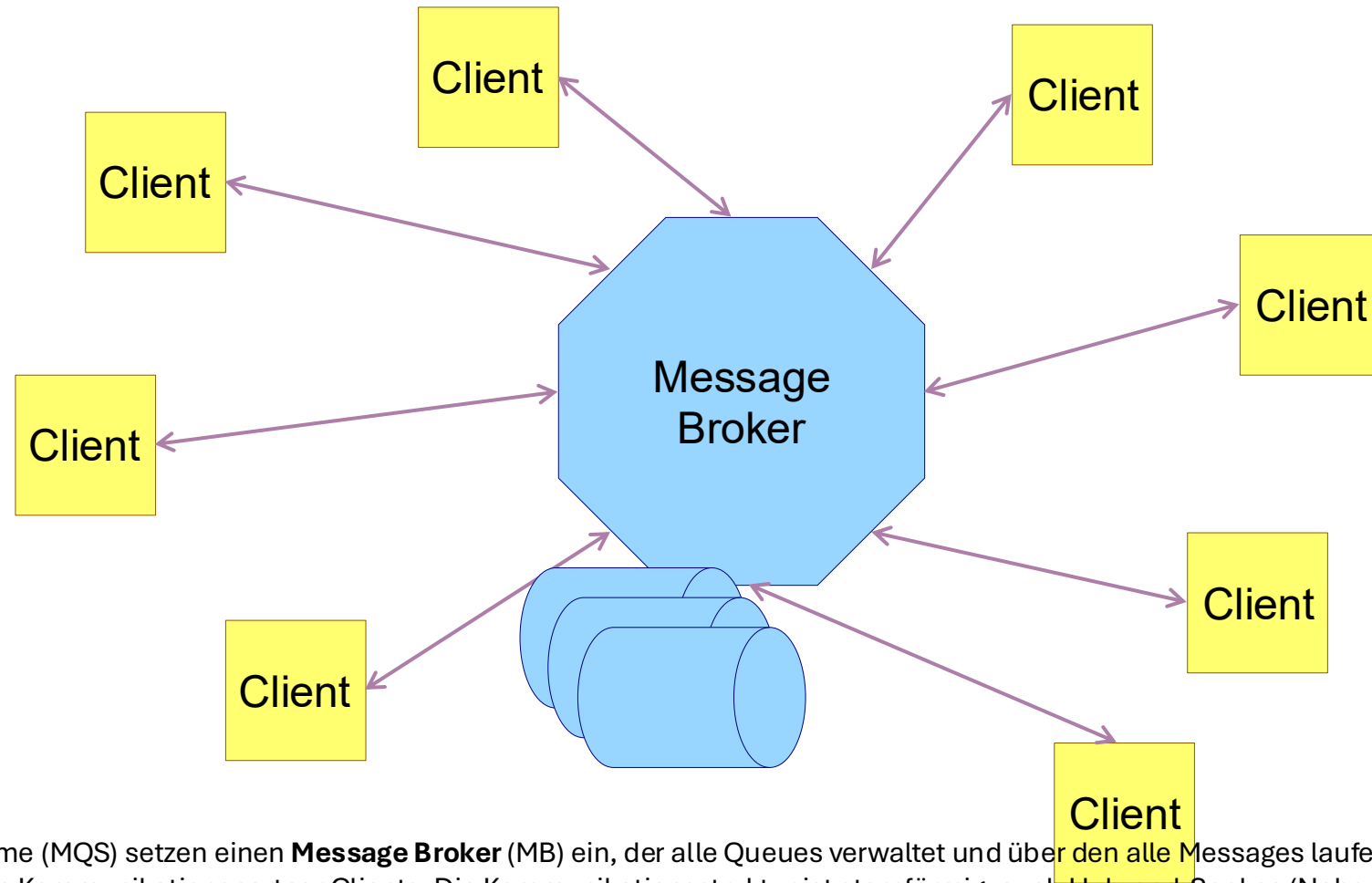
Publish / Subscribe (Topics)



Jede Nachricht hat ein Merkmal. Empfänger geben an, für welche Merkmale sie sich interessieren. Das MQS sorgt dafür, dass Nachrichten an die interessierten Empfänger gehen. Nachrichten, für die sich niemand interessiert, gehen verloren.

Beispiel: “order.created” (Interessenten: Lager, E-Mail, Rechnung)

Message Queueing (with Broker)



Viele Message Queueing Systeme (MQS) setzen einen **Message Broker** (MB) ein, der alle Queues verwaltet und über den alle Messages laufen. Aus Sicht des MB, d.h. der Server-Komponente des MQS, sind alle Kommunikationspartner Clients. Die Kommunikationsstruktur ist sternförmig, auch Hub-and-Spokes (Nabe und Speichen) genannt. Das ist unabhängig davon, welche Kommunikations-beziehungen mit Hilfe des MB zwischen dessen Clients implementiert werden. Es gibt auch MQS, die Kommunikation direkt und schneller abwickeln, zum Beispiel ZeroMQ. Die vorangegangenen Beziehungsbeispiele kann man auch damit implementieren. Die Partner sind dann eben keine Clients.

Zusammenfassung

- **Point to point:** Eine Queue
 - z.B. Load Balancing
- **Publish/Subscribe:** Mehrere Queues
 - z.B. Microservice-Interaktion
 - z.B. Event-driven architecture
- Verwaltung der Queues und Weiterleitung der Nachrichten durch **Message Broker**

Übung: Chatty (Verteilte Chat-Anwendung)

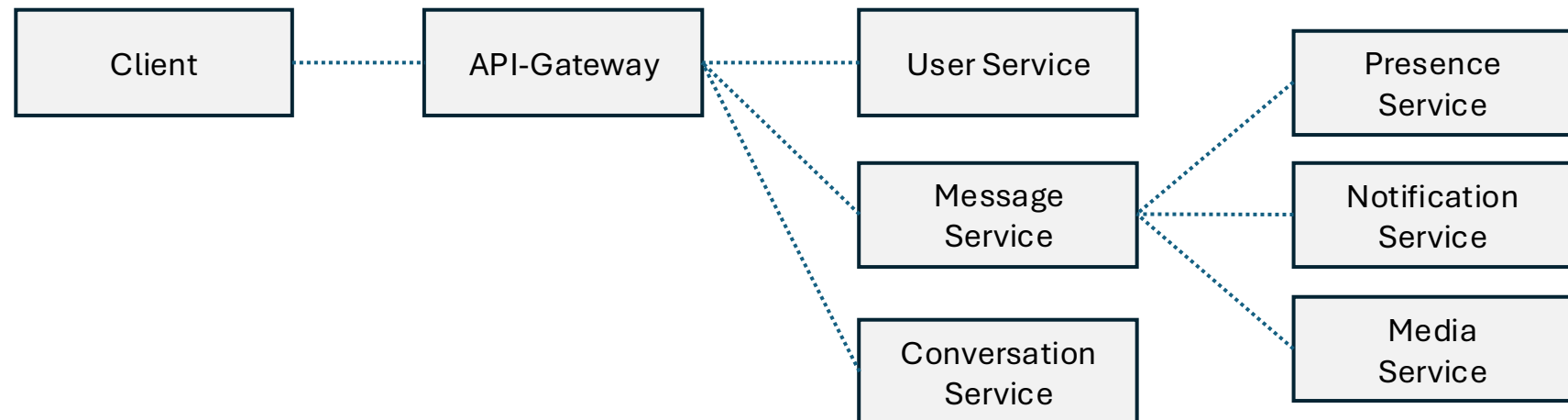


Teil 4: Asynchrone Kommunikation

Realisiert Chatty mithilfe eines Message Brokers (z.B. RabbitMQ).

- Welche Services interagieren für welche Use Cases mit dem Broker?
- Welche Prinzipien (Point-to-point, Publish/Subscribe) werden verwendet?
- Was gibt es zu beachten, wenn die Zahl der Nutzer „explodiert“?

Zur Erinnerung:



Event:

Producer:

Consumer:

① User_online
User_offline
⋮

Presence
Service

Notification
Service

Client
Service

User Service

② message_created

Message
Service

Notification
Service

Media
Service

Analysis
Service

Message Search
Service

③ media_uploaded

Media
Service

Notification
Service

④ message_read

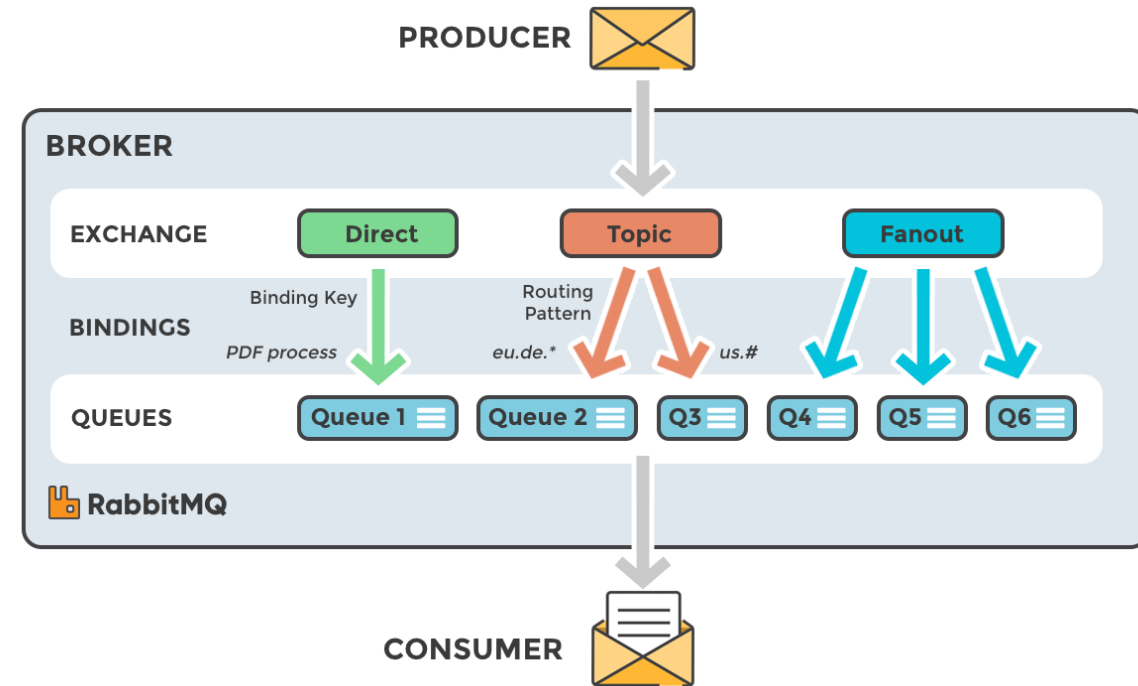
Client
Service

Notification
Presence
Conversation

RabbitMQ

Überblick RabbitMQ

- Open Source Message Broker (entwickelt in Erlang)
- Unterstützt verschiedene Protokolle (AMQP 0-9-1, AMQP 1.0, RabbitMQ Streams, MQTT, STOMP, WebSocket, HTTP)
- Unterstützt sowohl Point-to-point als auch Publish/Subscribe
- Kernelemente (aus AMQP)
 - **Exchange:** Erhält Nachrichten von Producern und entscheidet über Routing
 - **Queues:** Speichert Nachrichten, bis Consumer sie abholt
 - **Binding:** Regel, die Consumer mit einer Queue verknüpft
 - **Routing Key:** Nachrichtenattribut, basierend auf dem der Exchange entscheidet, welche Queue die Nachricht erhält



Quelle: <https://www.cloudamqp>

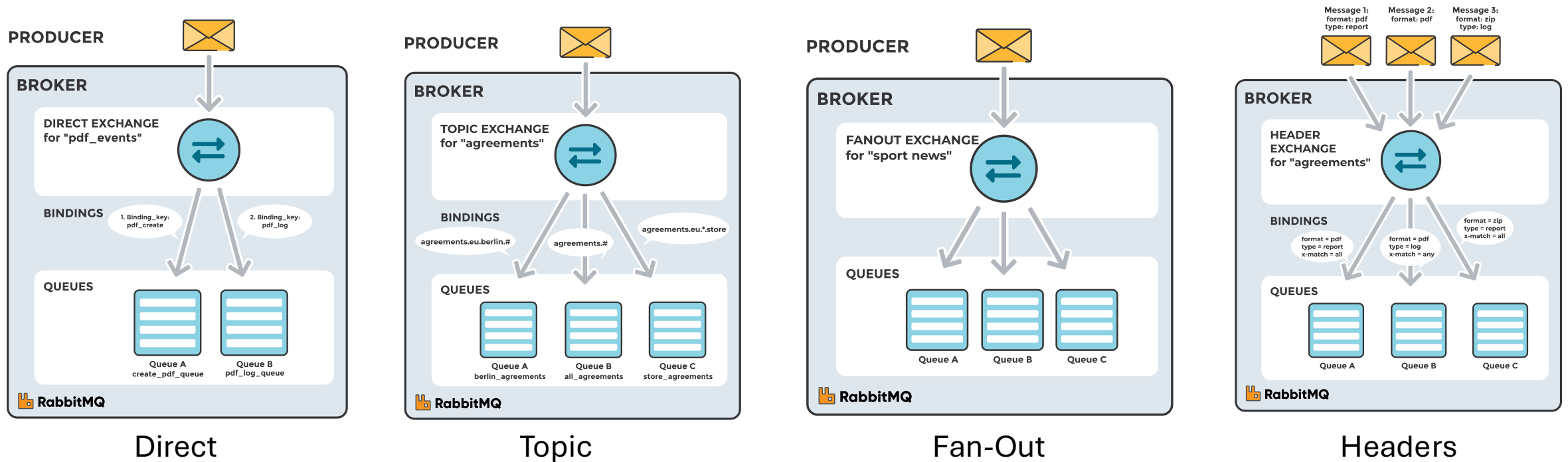
AMQP 0.9.1 Konzepte

<https://www.rabbitmq.com/tutorials/amqp-concepts>

Historische Entwicklung der AMQP-Versionen

https://en.wikipedia.org/wiki/Advanced_Message_Queueing_Protocol

Exchange-Arten in RabbitMQ

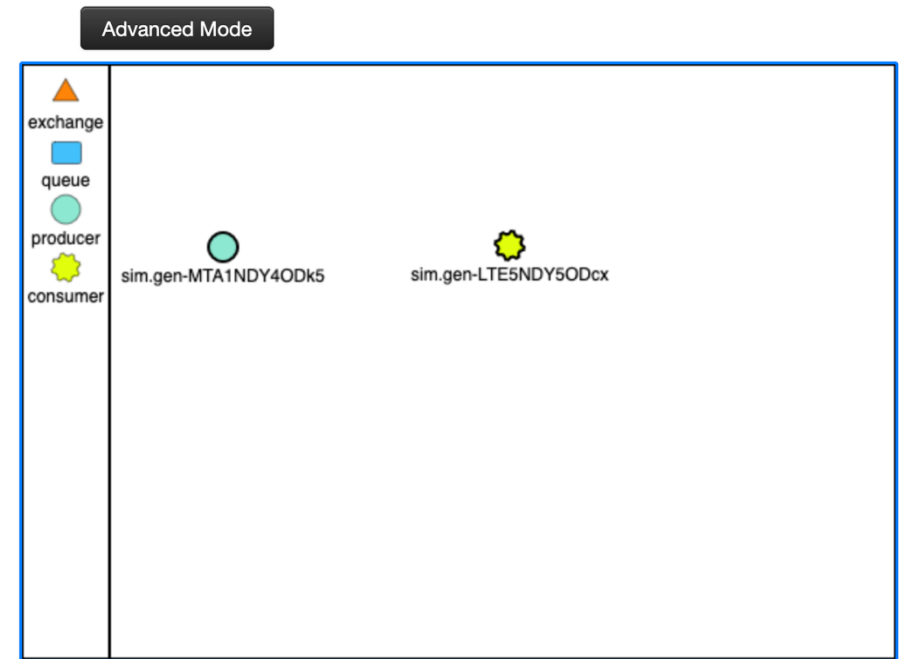


Demo

- <https://tryrabbitmq.com/>
 - Mehrere Exchanges und Exchange-Typen (Direct, Topic, Fan-Out)
 - Mehrere Queues möglich, auch Exchange als Queue

RabbitMQ Simulator

Use the drawing area below to represent your messaging topology. Drag messaging elements from the toolbox on the left to the canvas. To connect nodes, hold the ALT key (or SHIFT key) and drag from a source node to connect it to a destination node.





25 min

Übung

1. RabbitMQ unterstützt AMQP und MQTT. Wo liegt der Unterschied? Wann sollte welches Protokoll verwendet werden?
2. Welche Metadaten kann man in einer RabbitMQ-Nachricht spezifizieren?
3. Was ist das „Competing Consumers Pattern“? Was macht man, wenn man die gleiche Nachricht an mehrere Consumer senden möchte?
4. Wie lange werden Nachrichten in einer Queue gespeichert?

Lösungsskizze Aufgabe 1

MQTT: „eher IoT“
(lightweight)

- wenig Bandbreite (+)
- wenig Kontrolle (z.B. Filterlogik)
- keine klassischen Message Queues
- keine Rollbacks / Transaktionen
- weniger Enterprise-Features
(Prioritäten, TTL auf Queue-Ebene)

Lösungsskizze Aufgabe 2

ID

Content (z.B. content type, Encoding)

Ablaufinformation (TTL)

Timestamp (Sender)

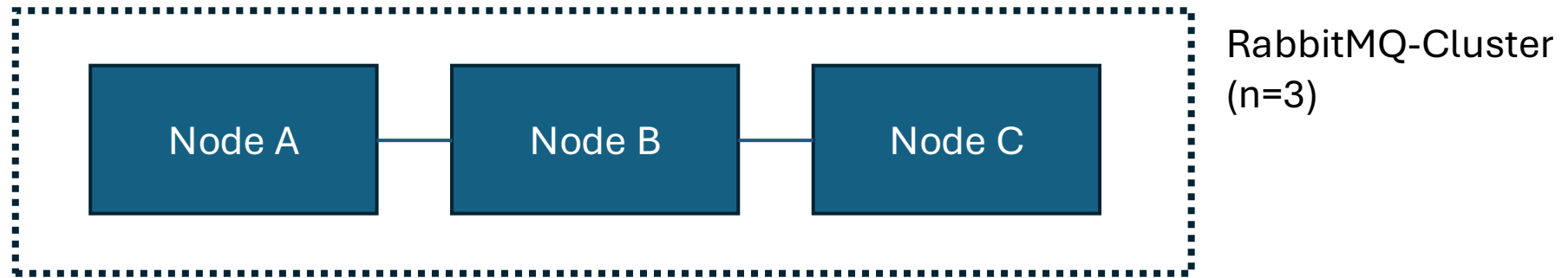
Delivery (transient, persistent)

Custom (key value pairs) → z.B. Trace ID

Nachrichtendauer in Queue:

- 1) Bis konsumiert ("Ack")
- 2) Queue voll
- 3) TTL erreicht
- 4) Queue gelöscht

Skalierung durch Clusterbetrieb



- Teilen von **Metadaten** (Exchanges, Queue-Definitionen, Bindings, Benutzer, Policies) durch verteilte Datenbank (Mnesia)
- **Nachrichten werden nicht repliziert** (jede Queue liegt auf einem Home-Node)

?

Was passiert, wenn der Home-Node ausfällt?

?

Distributed Consensus

Was ist „Distributed Consensus“?

Consensus (Konsens):

Bezeichnet allgemein den Zustand, dass sich mehrere Parteien auf einen einzigen Wert oder eine Entscheidung einigen.

Distributed Consensus (Verteilter Konsens):

Beschreibt spezifisch den Prozess, durch den unabhängige Knoten (Computer) in einem verteilten System (ohne zentrale Autorität) zu einer **Einigung über den Zustand von Daten** gelangen, selbst wenn einige Knoten ausfallen oder Fehler auftreten.

Anwendungen:

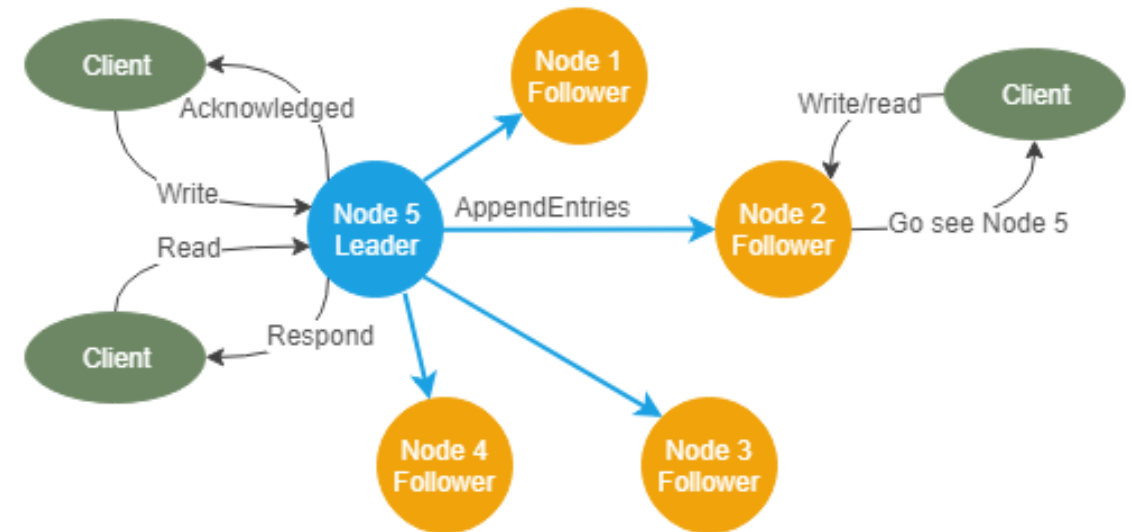
Blockchain-Technologien, verteilte Datenbanken, Cloud-Infrastruktur-Management, Leader Election in Clustern

Replikation in RabbitMQ

- Replikation von Nachrichten über „Quorum Queues“
 - Fehlertoleranz durch Nachrichtenreplikation
 - Hybridmodell möglich (Kombination von klassischen und Quorum Queues)
 - Raft Consensus-Mechanismus

Funktionsweise (Ausblick):

- Jede Queue besteht aus ungerader Zahl von Replikaten (Raft-Gruppe)
- Ein Knoten agiert als Leader, verwaltet Schreib- und Lesevorgänge
- Follower-Knoten replizieren die Daten
- Speicherung der Nachricht erfordert Bestätigung durch die Mehrheit



Verständnisfragen

Warum gehen beim Read-Anfragen beim Clusterbetrieb von RabbitMQ an den Leader?

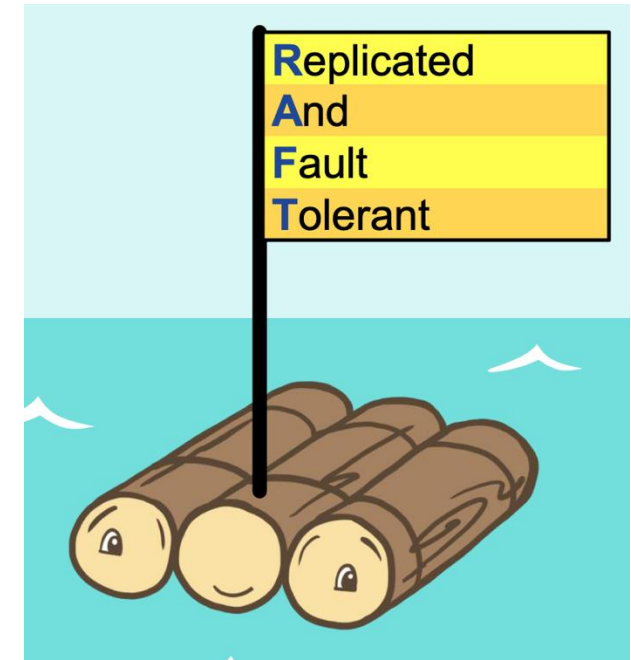
- Sicherstellung „**Strong Consistency**“
- Followers sind möglicherweise “hinter dem Leader“ und haben nicht den neuesten Stand
- Queue-Zustand muss zentral koordiniert werden

Wieso eine ungerade Anzahl von Knoten?

- Höhere Zahl der Knoten, die ausfallen dürfen, um immer noch Mehrheitsvotum durchzuführen.

The Raft Consensus Algorithm

- Original-Webseite:
<https://raft.github.io/>
(Beschreibung, Visualisierung, Vorträge, Code-Quellen):
- Original-Paper:
<https://classpages.cselabs.umn.edu/Spring-2018/csci8980/Papers/Consensus/Raft.pdf>
- Gute Zusammenfassung: <https://medium.com/@jitenderkmr/understanding-raft-algorithm-consensus-and-leader-election-explained-faadf28fd047>
- Wikipedia:
[https://en.wikipedia.org/wiki/Raft_\(algorithm\)](https://en.wikipedia.org/wiki/Raft_(algorithm))



Algorithms Should Be Designed For ...

Correctness?

Efficiency?

Conciseness?

Understandability!

Impact

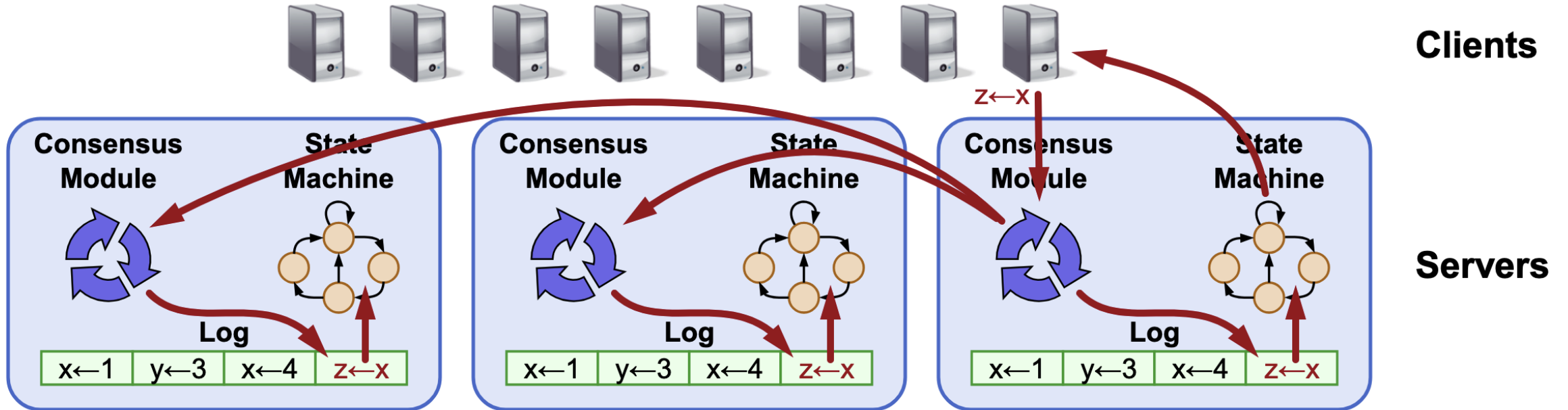
Hard to publish:

- **Rejected 3 times at major conferences**
- **Finally published in USENIX ATC 2014**
- **Challenges:**
 - PCs uncomfortable with understandability as metric
 - Hard to evaluate
 - Complexity impresses PCs

Widely adopted:

- **25 implementations before paper published**
- **83 implementations currently listed on Raft home page**
- **>10 versions in production**
- **Taught in graduate OS classes**
 - MIT, Stanford, Washington, Harvard, Duke, Brown, Colorado, ...

Replicated State Machine



- **Replicated log** ensures state machines execute same commands in same order
- Consensus module ensures proper log replication
- System makes progress as long as any majority of servers are up
- Failure model: delayed/lost messages, fail-stop (not Byzantine)

Paxos Protocol

- Leslie Lamport, 1989
- Nearly synonymous with consensus

“The dirty little secret of the NSDI community is that at most five people really, truly understand every part of Paxos ;-).” – NSDI reviewer

“There are significant gaps between the description of the Paxos algorithm and the needs of a real-world system...the final system will be based on an unproven protocol.” – Chubby authors

Kernprinzipien

Klare Rollenverteilung (Leader, Follower, Candidate)

- **Leader Election**

- Follower -> Candidate -> Leader

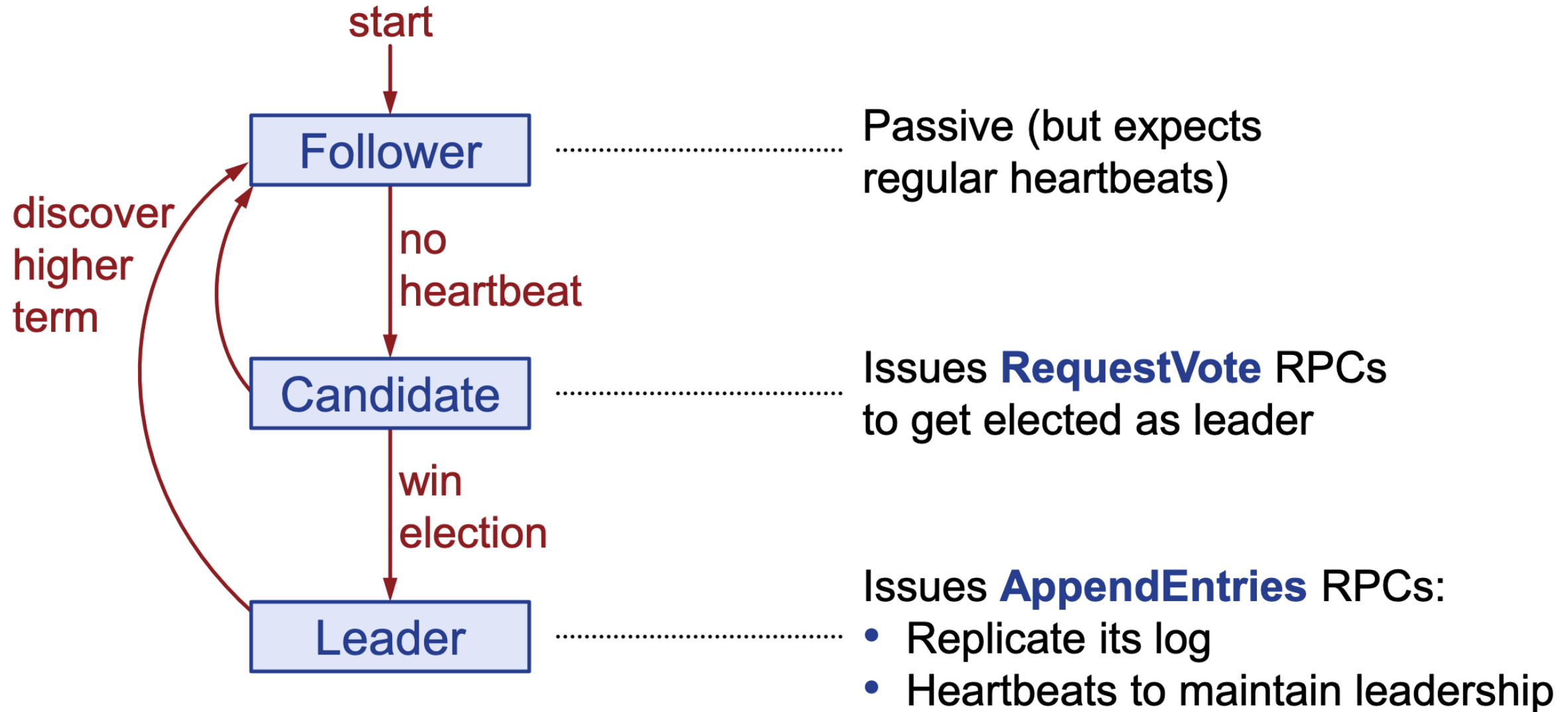
- **Log Replication**

- Leader repliziert Logs auf Follower
- Alle Änderungen gehen über den Leader, keine konkurrierenden Schreibzugriffe

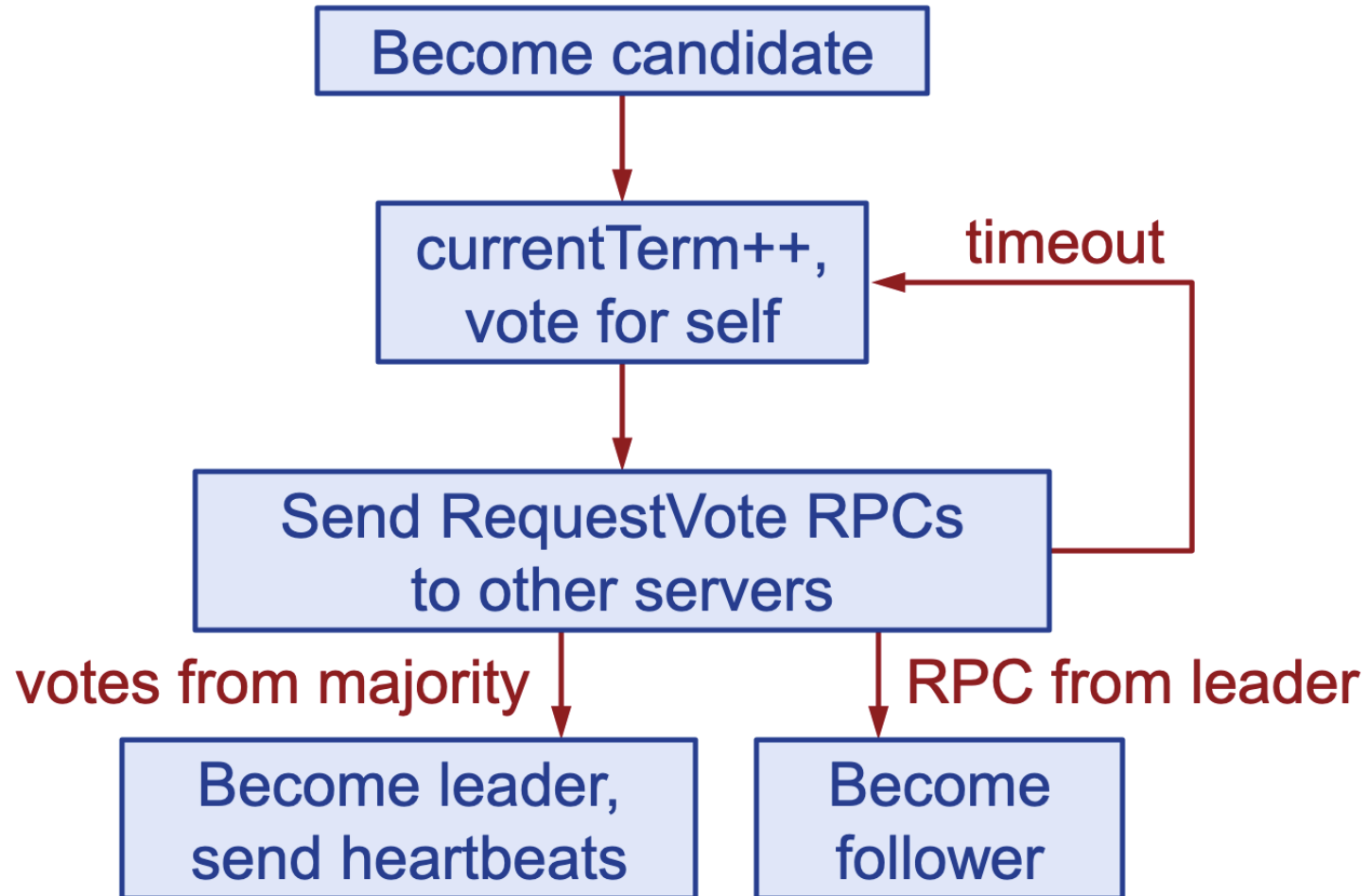
- **Safety**

- Mehrheit garantiert Konsistenz

Server States and RPCs



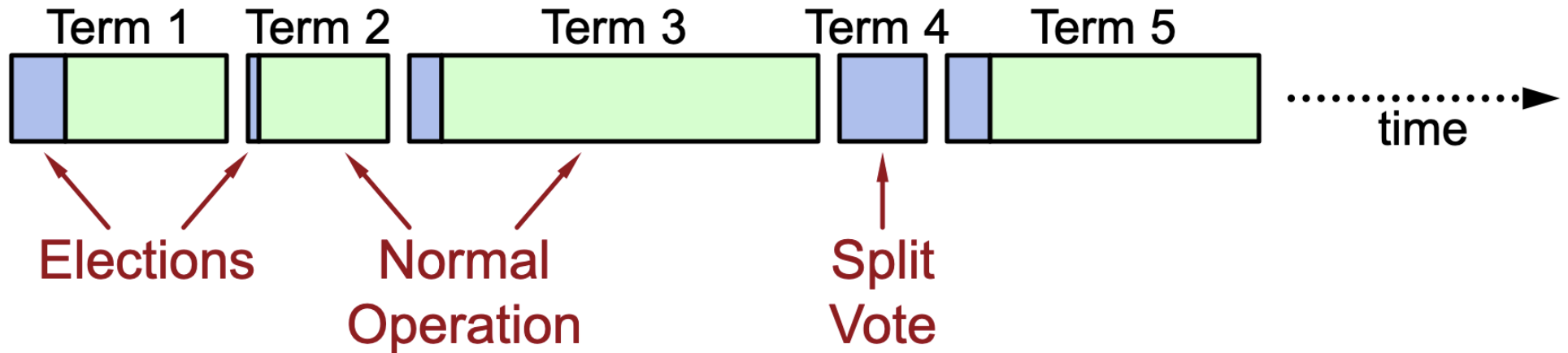
Leader Election



Normal Operation

- **Client sends command to leader**
- **Leader appends command to its log**
- **Leader sends AppendEntries RPCs to all followers**
- **Once new entry committed:**
 - Leader executes command in its state machine, returns result to client
 - Leader notifies followers of committed entries in subsequent AppendEntries RPCs
 - Followers execute committed commands in their state machines
- **Crashed/slow followers?**
 - Leader retries AppendEntries RPCs until they succeed
- **Optimal performance in common case:**
 - One successful RPC to any majority of servers

Terms



- At most 1 leader per term
- Some terms have no leader (failed election)
- Each server maintains **current term** value (no global view)
 - Exchanged in every RPC
 - Peer has later term? Update term, revert to follower
 - Incoming RPC has obsolete term? Reply with error

Terms identify obsolete information

Aufgabe

Was ist die Bedeutung der 3 Kernprinzipien von Raft? Wie werden sie vom Protokoll sichergestellt? Orientiert euch an folgenden Leitfragen:

- Leader Election
 - Wie stellt Raft sicher, dass ein Leader existiert? (Single Point of Failure)
 - Wie vermeidet Raft, dass zwei neue Leader gleichzeitig gewählt werden?
 - Was ist ein „Split Vote“ und was tut Raft dagegen?
- Log Replication
 - Wie stellt Raft sicher, dass Log-Einträge (in unserem Fall: Nachrichten in der Quorum Queue) in die ganze Gruppe repliziert werden?
 - Wann gilt eine Nachricht als „committed“?
 - Woher weiß ein Follower, dass er eine Nachricht verpasst hat?
- Safety
 - Wie stellt Raft sicher, dass Einträge, die „committed“ wurden, auch in der Queue bleiben? (Beispiel: Ein Follower, der Nachrichten verpasste, wird neuer Leader)
 - Raft ermöglicht „starke Konsistenz“ bei einer verhältnismäßig einfachen Umsetzung – gibt es einen Haken?

Zusammenfassung: Raft

Einfaches Protokoll mit „starker Konsistenz“ trotz möglichem Nachrichtenverlust und Serverausfall – falls alle Server „ehrlich“ sind

- Mehrheits-basiertes Commitment
→ Eintrag wird gegenüber dem Client nur committed, wenn Mehrzahl der Server akzeptiert hat
- Leader-gesteuerte Replikation
→ Vermeidung von Schreibkonflikten
- Nur Knoten mit aktuellen Logs können Leader werden
→ Follower wählt nur für Candidate, falls Candidate mindestens genauso aktuell ist (es gehen keine Daten verloren, falls Follower offline war)

